Notch Activities

This document is the implementation contract for source-defined activities in Boring Notch. It describes the code that exists in this repository, the three presentations every new activity must provide, and the design and test requirements for accepting an activity.

An activity is Swift code compiled into the app. It is not an ActivityKit activity, extension, script, package, or runtime plugin. Adding an activity requires changing the app source and Xcode project.

Product contract

Every new activity must provide all three of these product surfaces:

Product surface	Code surface	Required behavior
Its own tab in the open notch	`makeExpandedView()` and registration in `ActivityRegistry.shared`	A complete, independently useful interface selected through `.activity(id)`
Its own complete chin presentation	`makeLivePresentationView()`	The full closed-notch presentation used when it is the only selected live provider
Its own one-side chin presentation	`makeMinimalLivePresentationView()`	A minimal presentation that fits on either side when another live provider is also visible

In this document, **chin** means the content placed around the physical camera/notch while Boring Notch is closed. The code calls these surfaces **live presentations**. A **complete chin** maps to `ActivityLivePresentationStack.full`; a **one-side chin** maps to one side of `ActivityLivePresentationStack.split`.

The protocol currently supplies defaults that let an activity compile without live presentations. That is a framework convenience, not the acceptance standard. A new activity is incomplete until it has a dedicated tab, an explicit full chin view, and an explicit minimal one-side chin view. `makeCompactView()` does not satisfy either chin requirement because the production closed-notch renderer never calls it.

An activity does not need to remain visible on the chin at all times. It must provide both chin layouts, then use `livePresentationState` to show them only while there is meaningful, current information to display.

Architecture

The open-notch and closed-notch paths share the same registered activity instance:

```
Concrete NotchActivity
    -> AnyNotchActivity
        -> ActivityRegistry
            -> open-notch tabs, pagination, swipe navigation, settings lookup
        -> AnyLiveActivityPresentationProvider
            -> LiveActivityPresentationProviderRegistry
                -> ActivityLivePresentationCoordinator
                    -> full or split closed-notch chin rendering
```

`NotchActivity` uses associated view types so concrete activities can return `some View`. `AnyNotchActivity` erases those types only at the heterogeneous registry boundary and forwards the activity's `objectWillChange`. The live-provider adapter then exposes registered activities to the closed-notch stack.

The important implementation files are:

File	Responsibility
`boringNotch/activities/NotchActivity.swift`	IDs, metadata, protocols, type erasure, live-provider adapters, selection, recency, sizing, expanded/configuration hosts
`boringNotch/activities/ActivityRegistry.swift`	Production registration, duplicate-ID validation, order, availability, and active filtering
`boringNotch/ContentView.swift`	Open destination rendering, closed-notch interruption precedence, chin layout, hover routing, and window width
`boringNotch/components/Tabs/TabSelectionView.swift`	Tab models, visible page order, fallback resolution, and pagination dots
`boringNotch/extensions/PanGesture.swift`	Horizontal swipe routing over the same visible page order
`boringNotch/sizing/matters.swift`	Open-notch bounds, preferred-height propagation, and closed-notch measurements
`boringNotch/components/Settings/SettingsView.swift`	Explicit settings navigation and activity configuration hosts
`boringNotchTests/ActivityArchitectureTests.swift`	Registry, observation, selection, recency, sizing, provider, and interruption contracts

Concrete examples are split by responsibility:

Activity	Adapter	UI and state
Calendar	`activities/CalendarActivity.swift`	`components/Calendar/BoringCalendar.swift`, `managers/CalendarManager.swift`, calendar models/providers
Pomodoro	`activities/PomodoroActivity.swift`	`components/Pomodoro/PomodoroActivityView.swift`, `managers/PomodoroManager.swift`, `models/PomodoroSession.swift`
Example	`activities/ExampleActivity.swift`	Minimal compile-time example; intentionally not registered

`NotchActivity` contract

All activity APIs are `@MainActor`. A conforming class is an `ObservableObject` and has these associated view types:

- `ExpandedContent`: required dedicated-tab content.
- `CompactContent`: optional alternate content; defaults to `EmptyView` and is not currently mounted by production UI.
- `LivePresentationContent`: required by the product contract for the complete chin, although the protocol defaults it to `EmptyView`.
- `MinimalLivePresentationContent`: required by the product contract for the one-side chin. The protocol can reuse `LivePresentationContent`, but new activities must design and implement a space-appropriate minimal view explicitly.
- `ConfigurationContent`: optional settings content; defaults to `EmptyView`.

Identity and metadata

Every activity needs a permanent `ActivityID` and immutable `ActivityMetadata`:

```
extension ActivityID {
    static let delivery = ActivityID("community.example.delivery")
```

```
}

let id = ActivityID.delivery
let metadata = ActivityMetadata(
    name: String(localized: "Delivery"),
    systemImage: "shippingbox.fill",
    tint: .blue,
    preferredExpandedHeight: openNotchSize.height
)
```

Use `builtin.<activity>` for app-owned additions and `community.<publisher>.<activity>` for contributed additions. The existing `calendar` ID predates this convention and is the only legacy exception. Never derive an ID from a localized name, reuse `builtin.time` or `builtin.media`, or change a released ID. `ActivityRegistry` rejects duplicate registered-activity IDs, but `LiveActivityPresentationProviderRegistry` does not validate collisions with Timer or Media; a collision corrupts recency and routing because snapshots are keyed by `ActivityID`.

Metadata drives the tab label, tab SF Symbol, selected-tab tint, automatic chin accessory, accessibility fallback name, and optional expanded height. `AnyNotchActivity` captures `id` and `metadata` when it is initialized, so treat both as immutable.

`preferredExpandedHeight` is optional. The normal open height is 190 points, Calendar requests 210 points, and `clampedOpenNotchHeight` constrains all preferences to 190...300 points. The window height adds 20 points of shadow space. An activity must adapt within the 640-point open-notch width and this height range rather than resizing the window itself.

State properties

Property	Default	Actual consumer
`isAvailable`	`true`	Tab/pagination/swipe inclusion and registered live-provider eligibility
`isActive`	`false`	`ActivityRegistry.activeActivities` only
`supportsCompactPresentation`	`false`	Capability metadata only; production UI does not currently mount the compact view
`livePresentationState`	`.hidden`	Closed-notch eligibility and visible/hidden transitions
`livePresentationSizing`	Full 64, minimal 56	Closed-notch rendering and window width calculation
`supportsConfiguration`	`false`	`ActivityConfigurationView`

`isActive` and chin visibility are intentionally independent. Starting work normally sets `isActive == true` and returns a visible `livePresentationState`, but only the latter enters the chin selector. Do not expect `isActive` to show UI automatically.

Any property that changes availability, active state, live state, or rendered data must emit `objectWillChange`. Use `@Published` for state owned by the activity. If a manager owns the state, subscribe to the manager and forward its change notification, as `PomodoroActivity` does. The type-erased activity forwards that notification to the registry, navigation, and live-provider registry.

Registration creates the dedicated tab

Production activities are registered in `ActivityRegistry.shared`:

```
return try ActivityRegistry {
    CalendarActivity()
    PomodoroActivity()
```

```
        DeliveryActivity()
    }
```

No separate edit to `TabSelectionView` is needed. Registration automatically contributes `.activity(activity.id)` to all of these locations when `isAvailable` is true:

- The tab strip in `BoringHeader`.
- The open-notch destination switch in `ContentView`.
- Pagination dots.
- Horizontal two-finger navigation.
- Live-activity hover routing back to the activity's tab.

`BoringHeader` can hide the entire tab strip through the global `alwaysShowTabs` preference, but the activity remains a distinct destination in pagination and swipe navigation. An activity must not add a second, private tab control to work around that global choice.

The current page order is Home, each available registered activity in registry order, the legacy Activities page, and Shelf when Shelf is enabled. The legacy `.activities` page is `TimeActivityView` for Timer and Stopwatch; it is not a registered `NotchActivity` and must not be used as the destination for new custom activities.

When an activity becomes unavailable, `ContentView` recomputes the visible pages. If the current destination no longer exists, `resolvedNotchView` returns Home. Availability therefore must represent whether the destination should exist—not temporary loading or an empty result. A missing permission should normally keep the activity available and show recovery UI in its tab; hide it only when the product explicitly disables the entire feature.

Registration is source-defined and fixed for the process lifetime. Changing `isAvailable` can hide a registered entry, but activities cannot be dynamically added to or removed from the registry at runtime.

`ActivityRegistryBuilder` supports conditionals, optionals, and arrays for source composition. Those branches run only while the registry is constructed; they are not a runtime installation mechanism. Prefer registering the activity once and publishing `isAvailable` when visibility must change while the app runs.

Dedicated open-notch presentation

`ContentView` resolves `.activity(id)` through the registry and mounts `ExpandedActivityView`. That host:

- Calls `makeExpandedView()`.
- Applies `metadata.preferredExpandedHeight` through `preferredOpenNotchHeight`.
- Calls `activityDidAppear()` and `activityDidDisappear()`.
- Is recreated when `coordinator.currentView` changes because the destination container is keyed by the current view.

The expanded view must be the complete activity experience. Put controls, detailed status, errors, empty states, and permission recovery here. Do not require the chin to operate the feature.

Activity instances are shared singletons through `ActivityRegistry.shared`, while the app can create one notch window per display. Appearance callbacks can therefore run once per visible window and may overlap. Make them idempotent or reference-counted. Prefer a view's own `onAppear` and `onDisappear` for view-local work; use activity callbacks only when the activity object itself owns visibility-scoped work.

Complete and one-side chin presentations

Eligibility

A registered activity is adapted to `LiveActivityPresentationProvider`. It is eligible only when both conditions are true:

1. `isAvailable == true`.
2. `livePresentationState` is `.visible(priority: ...)`.

Use `.hidden` when the task has not started, has ended, or has no useful glanceable state. Use `.visible(.normal)` while progressing and `.visible(.low)` for a meaningful paused state. `.high` exists, but priority does not currently affect selection; all three visible priorities are equally eligible.

Complete chin: one selected provider

When exactly one provider is eligible, the selector returns `.full(provider)`. The host renders:

```
[automatic accessory] [physical-notch spacer] [activity full content]
```

For a registered activity, the accessory is automatically created from `metadata.systemImage`, set at 16-point semibold, tinted with `metadata.tint`, and hidden from accessibility because the full content labels the combined presentation. `makeLivePresentationView()` supplies only the activity content on the opposite side. Do not repeat the metadata icon in that view.

The default full-content width is 64 points. Override `livePresentationSizing.fullContentWidth` only when the content has a deterministic need. The host fixes the returned view to that width, so text must use a stable format, one line, a minimum scale factor where appropriate, and monospaced digits for changing numbers.

One-side chin: two selected providers

When two or more providers are eligible, only the two selected providers render. Each uses `makeMinimalLivePresentationView()` on one side of the physical notch. The older selected provider is placed on the leading side and the newest on the trailing side. For registered activities the host adds the metadata accessory beside the minimal content, nearest the physical notch.

The default minimal-content width is 56 points. An explicit width of zero produces an icon-only presentation because the registered-activity adapter still supplies its accessory. Icon-only is technically supported, but Apple recommends updated information instead of only a logo when that information can remain recognizable and legible. Prefer a short value such as remaining time, progress, score, or state. Do not squeeze the complete-chin layout into the one-side slot.

`makeCompactView()` is unrelated to this path. `supportsCompactPresentation` and `PomodoroCompactView` currently have no production rendering consumer.

Width calculation

The shell owns physical-notch spacing and window sizing. Activities publish content widths, not total widths.

At render time:

- `accessorySize = max(0, effectiveClosedNotchHeight - 12)`.
- Full additional width is `accessorySize + fullContentWidth + 20`.
- A minimal registered activity uses `accessorySize + 6 + minimalContentWidth`; the 6-point gap disappears when minimal content width is zero.
- Split additional width is both minimal provider widths plus 20.
- `.accessorySize` can be used instead of a fixed content width when content must match the accessory dimensions.

The same selected stack calculates the window width and renders the UI. An activity must not read the screen notch width, insert its own physical-notch spacer, mutate `BoringViewModel.notchSize`, or use geometry feedback to resize the shell.

Recency and selection

`ActivityLivePresentationCoordinator` observes the unified provider registry and assigns an in-memory sequence when a provider changes from ineligible to eligible. The selector filters eligible providers and sorts by that start sequence, not by priority.

- Zero eligible providers: render no activity stack and preserve the normal idle fallback.
- One eligible provider: render its full presentation.

- Two or more eligible providers: render the two most recently started using their minimal presentations.
- A visible priority change, such as normal to low on pause, preserves recency.
- Hiding, ending, or becoming unavailable removes the sequence and promotes the next eligible provider.
- Becoming eligible again receives a new sequence.
- On launch, already-eligible providers have no sequence. Registry order is the deterministic fallback; the first two eligible providers win, with the second on the leading side and the first on the trailing side.

The production provider order is all registered activities, followed by the Timer adapter and Media adapter. Recency is not persisted across launches.

Hover and opening behavior

Hovering an activity accessory or live-content area prepares that activity's `.activity(id)` destination when `openNotchOnHover` is enabled. The normal notch hover handler then opens the shell after the configured delay. Timer routes to the legacy `.activities` page and Media routes to Home. Hovering the physical-notch center can route to Home when `openMediaTabOnChinHover` is enabled.

The chin is a glanceable shortcut, not the primary control surface. Keep destructive actions, multi-step interactions, permission requests, and dense controls in the dedicated tab.

Transient interruptions

The selected stack remains intact while higher-precedence transient content temporarily replaces or overlays it. Current interruption types are startup, battery, Bluetooth, system HUD, legacy media notification, and Timer completion. They do not change eligibility or recency. A custom activity must tolerate disappearing briefly and must derive its current display from source state when it returns.

State, persistence, and performance

Use the same state model for the tab, complete chin, and one-side chin. Creating separate state for each presentation causes drift.

- Use `@State` for state owned by one rendered view.
- Use `@StateObject` when that rendered view owns a reference model.
- Use `@ObservedObject` for an injected/shared manager.
- Use a manager for persistent sessions, permissions, services, system observers, and state shared between presentations or displays.

Persistent time-based activities should store timestamps and a validated `Codable` snapshot, not accumulated UI ticks. Pomodoro and Timer calculate elapsed/remaining time from timestamps, persist snapshots in `UserDefaults`, reconcile after wake/app activation, and schedule only the next meaningful completion. Their `TimelineView` refreshes only while running and pauses while static.

Keep activity and registry initialization cheap. Do not request permission, start high-frequency polling, or perform blocking I/O in the activity initializer. Start expensive work on explicit user action or managed lifecycle demand, and stop it when no longer needed. The registry holds activities for the whole app process.

Configuration

Configuration is opt-in:

```
var supportsConfiguration: Bool { true }

func makeConfigurationView() -> some View {
    DeliverySettingsView()
}
```

`ActivityConfigurationView(activityID:)` looks up the activity and mounts its configuration only when `supportsConfiguration` is true. Registration does not create a Settings sidebar entry. Add the `NavigationLink` and detail switch case in `SettingsView` explicitly, add persistent keys in `models/Constants.swift`, and localize user-facing strings.

Configuration changes must define whether they affect the active session or only the next session. Pomodoro, for example, snapshots the current duration and applies duration-setting changes to the next session.

Complete implementation example

This example shows the required adapter shape. The manager and three concrete views remain separate files in production code.

```
import Combine
import SwiftUI

extension ActivityID {
    static let delivery = ActivityID("community.example.delivery")
}

@MainActor
final class DeliveryActivity: NotchActivity {
    static let activityID = ActivityID.delivery

    let id = activityID
    let metadata = ActivityMetadata(
        name: String(localized: "Delivery"),
        systemImage: "shippingbox.fill",
        tint: .blue
    )

    private let manager: DeliveryManager
    private var managerObservation: AnyCancellable?

    init(manager: DeliveryManager? = nil) {
        self.manager = manager ?? .shared
        managerObservation = self.manager.objectWillChange.sink { [weak self] _ in
            self?.objectWillChange.send()
        }
    }

    var isAvailable: Bool { manager.isConfigured }
    var isActive: Bool { manager.currentDelivery != nil }

    var livePresentationState: ActivityLivePresentationState {
        guard manager.currentDelivery != nil else { return .hidden }
        return manager.isPaused
            ? .visible(priority: .low)
            : .visible(priority: .normal)
    }

    let livePresentationSizing = LiveActivityPresentationSizing(
        fullContentWidth: .fixed(64),
        minimalContentWidth: .fixed(40)
    )

    var supportsConfiguration: Bool { true }

    func makeExpandedView() -> some View {
        DeliveryActivityView(manager: manager)
    }

    // Complete chin content. The host supplies the icon on the other side.
```

```
    func makeLivePresentationView() -> some View {
        DeliveryFullChinView(manager: manager)
    }

    // One-side chin content. The host supplies the adjacent icon.
    func makeMinimalLivePresentationView() -> some View {
        DeliveryMinimalChinView(manager: manager)
    }

    func makeConfigurationView() -> some View {
        DeliverySettingsView()
    }
}
```

After creating the adapter:

1. Add the model, manager/service, expanded view, full chin view, minimal chin view, and optional settings view.
2. Add every Swift file to the `boringNotch` app target in `boringNotch.xcodeproj`.
3. Register exactly one activity instance in `ActivityRegistry.shared`.
4. Add localized strings and any Defaults keys.
5. Add an explicit Settings route when configuration is supported.
6. Add focused architecture, manager, persistence, and navigation tests.

The app deployment target is macOS 14. Verify every API and SF Symbol against that target even when development uses a newer Xcode or macOS release.

Existing implementation audit

Feature	Dedicated tab	Complete chin	One-side chin	Notes
Pomodoro	Yes, `.activity(.pomodoro)`	Remaining time and paused state	Automatic red Timer accessory; custom minimal content is currently empty	Meets the three code paths, but showing a short dynamic value in the minimal view would align more closely with Apple's minimal-presentation guidance
Calendar	Yes when `showCalendar` is enabled	No	No	Registered and configurable, but expanded-only; it does not meet the required three-presentation contract yet
Example	Only if temporarily registered	No	No	Demonstrates basic type conformance and compact capability; not production registered and not a complete template for a new activity
Timer/Stopwatch	Shared legacy `.activities` page, not a registered activity	Yes through `TimeLiveActivityProvider`	Accessory only	Additional provider, ID `builtin.time`; visibility also depends on `clockShowInClosedNotch`

Feature	Dedicated tab	Complete chin	One-side chin	Notes
Media	Home, not a registered activity	Artwork plus visualization	Artwork	Additional provider, ID `builtin.media`; remains eligible during its configured post-pause grace period

Battery, Bluetooth, HUDs, downloads, Timer completion, `BatteryActivityManager`, and `components/Live activities/LiveActivityModifier.swift` are not part of the custom activity registry. `ActivityType` in that legacy modifier is unrelated to `ActivityID` and must not be used to create a new activity.

Apple design requirements

This app implements a custom macOS notch surface, not the system Dynamic Island. Even so, activity design must follow the applicable official guidance in Apple's Live Activities, Designing for macOS, Layout, Buttons, Typography, Color, SF Symbols, Motion, and Accessibility guidance.

These are acceptance requirements:

Information and hierarchy

- Use a live presentation only for a task or event with a clear current state and a meaningful beginning/end or active/inactive boundary.
- Show only the most important glanceable value on the chin. Put detail and controls in the dedicated tab.
- Keep the complete and one-side presentations recognizably related through consistent vocabulary, symbol, color, and value formatting.
- Make the one-side presentation independently recognizable. Prefer current information over a static symbol when it remains legible.
- Never show ads, promotions, decorative status with no user value, or sensitive details that a nearby observer should not see.

Layout

- Respect the physical camera/notch and let the host own its spacer, outer shape, corner radii, margins, and window size.
- Keep chin content as narrow as the information permits. Do not add padding next to the camera housing or draw content to the outer rounded edge.
- Use deterministic full and minimal widths and verify long localized text, large values, and both physical-notch and non-notch displays.
- Preserve relative information placement between full and minimal presentations so the transition is predictable.
- Use the 640-by-190-point open layout as the baseline and request extra height only for content that needs it; never exceed the host's 300-point clamp.

Typography, symbols, and color

- Use system text styles or SF Pro through `.system`; avoid custom typefaces in the chin.
- Apple lists 13 points as the macOS default and 10 points as the minimum. Chin-critical text should normally be at least 13 points with medium or semibold weight.
- Use monospaced digits and numeric content transitions for changing measurements and timers.
- Use an SF Symbol that exists on the macOS 14 deployment target. The symbol must communicate the activity without depending on tint.
- Prefer semantic/system colors, maintain sufficient contrast on black, and communicate state with text or symbols in addition to color.

Interaction and accessibility

- Use real `Button`, `Toggle`, `Picker`, and other semantic controls in the expanded tab. Do not implement primary actions with tap gestures on unlabeled shapes.

- Give every icon-only control a concise accessibility label and macOS help/tooltip. Label changing chin content as one combined, meaningful value rather than exposing decorative children.
- Provide visible enabled, disabled, pressed, running, paused, completed, empty, loading, permission-denied, and error states as applicable.
- Support keyboard and VoiceOver operation in the dedicated tab. Hover-to-open is an optional shortcut, not the only route.
- Do not rely on color, animation, sound, or haptics alone to communicate state. Audit the activity with Accessibility Inspector and increased-contrast settings.

Motion and efficiency

- Animate state changes to explain continuity, not to attract attention. Avoid perpetual decorative animation in the chin.
- Respect Reduce Motion and replace large scale/translation effects with a restrained fade or no animation.
- Pause timelines and animations when values are static or hidden. Use the slowest refresh interval that displays correct information.
- Reconcile from timestamps after sleep, wake, app activation, and delayed execution instead of assuming periodic timers ran on schedule.

Test and review requirements

At minimum, add tests covering:

- Stable, unique ID and metadata.
- Default registration and registry lookup.
- Dedicated-tab insertion in `visibleNotchViews` and fallback when unavailable.
- `objectWillChange` propagation from the manager through `AnyNotchActivity` and `ActivityRegistry`.
- `isActive` transitions independently of `livePresentationState`.
- Hidden, running, paused, completed, reset, unavailable, and reactivated live states.
- One eligible provider selecting `.full`.
- Two eligible providers selecting `.split` with the expected leading/trailing order.
- Three or more providers promoting the next most recent after one ends.
- Deterministic full/minimal width calculations, including the real activity's chosen widths.
- Persistent snapshot validation and timestamp-derived restoration, when state persists.
- Configuration support and the rule for changes during an active session.

Run the macOS test target:

```
xcodebuild test \
  -project boringNotch.xcodeproj \
  -scheme boringNotch \
  -destination 'platform=macOS'
```

Manual review must cover open and closed notch states, one/two/three simultaneous providers, all interruption types, physical-notch and non-notch screens, multiple displays, hidden-in-full-screen behavior, chin hover enabled/disabled, long localization, keyboard-only control, VoiceOver, Increase Contrast, and Reduce Motion.

Acceptance checklist

An activity is ready only when every applicable item is true:

- It has a permanent namespaced `ActivityID` that does not collide with registered activities, Timer, or Media.
- It is registered once and automatically receives its own available tab, page dot, and swipe destination.
- Its expanded tab is complete and usable without the chin.
- It explicitly implements both `makeLivePresentationView()` and `makeMinimalLivePresentationView()`.
- It publishes a deliberate hidden/running/paused/ended `livePresentationState`.
- Its full and minimal widths are deterministic and tested.
- All presentations observe one source of truth and recover after interruption or app sleep.

- Its settings, persistence, permissions, localization, errors, and empty states are defined.
- It meets the Apple design and accessibility requirements above.
- Automated tests pass and the full manual presentation matrix has been reviewed.